Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ДОМАШНЯЯ РАБОТА 1

«Реализация алгоритма поиска с опечатками»

ДИСЦИПЛИНА: «Архитектура Автоматизированных Систем Обработки Информации и Управления»

Выполнил: студент гр. ИУ5-24Б _____ (Олейник В. Д.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю. Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Цели:

- Реализовать алгоритм нахождения расстояния Левенштайна
- Реализовать алгоритм нахождения расстояния Дамерау-Левенштайна
- Реализовать оптимальный алгоритм нахождения расстояния Дамерау-Левенштайна
- Сравнить время работы оптимальной реализации и обычной, для расстояния Дамерау-Левенштайна

```
int LevenshteinDistance(string s1, string s2)
{
    s1 = s1.ToUpper();
    s2 = s2.ToUpper();

    int m = s1.Length;
    int n = s2.Length;

    int[,] d = new int[m + 1, n + 1];

    for (int i = 0; i ≤ m; i++) d[i, 0] = i;
    for (int j = 0; j ≤ n; j++) d[0, j] = j;

    for (int i = 1; i ≤ m; i++)
    {
        for (int j = 1; j ≤ n; j++)
        {
            int cost = (s1[i - 1] == s2[j - 1]) ? 0 : 1;
            int delete = d[i - 1, j] + 1;
            int insert = d[i, j - 1] + 1;
            int replace = d[i - 1, j - 1] + cost;
            d[i, j] = Math.Min(Math.Min(delete, insert), replace);
        }
    }

    return d[m, n];
}
```

рис. 1 – реализация поиска расстояния Левенштайна

```
1 int DamerauLevenshteinDistance(string s1, string s2)
2 {
3     s1 = s1.ToUpper();
4     s2 = s2.ToUpper();
5
6     int m = s1.Length;
7     int n = s2.Length;
8
9     int[,] d = new int[m + 1, n + 1];
10
11    for (int i = 0; i ≤ m; i++) d[i, 0] = i;
12    for (int j = 0; j ≤ n; j++) d[0, j] = j;
13
14    for (int i = 1; i ≤ m; i++)
15    {
16        for (int j = 1; j ≤ n; j++)
17        {
18            int cost = (s1[i - 1] == s2[j - 1]) ? 0 : 1;
19
20            int delete = d[i - 1, j] + 1;
21            int insert = d[i, j - 1] + 1;
22            int replace = d[i - 1, j - 1] + cost;
23
24            d[i, j] = Math.Min(Math.Min(delete, insert), replace);
25
26            if (i > 1 && j > 1 &&
27                s1[i - 1] == s2[j - 2] &&
28                s1[i - 2] == s2[j - 1])
29            {
30                int transpose = d[i - 2, j - 2] + cost;
31                d[i, j] = Math.Min(d[i, j], transpose);
32            }
33        }
34    }
35
36    return d[m, n];
37 }
```

рис. 2 – реализация алгоритма Дамерау-Левенштайна

```
int DamerauLevenshteinOptimized(string s1, string s2)
{
    s1 = s1.ToUpper();
    s2 = s2.ToUpper();

    if (s1.Length < s2.Length)
        (s1, s2) = (s2, s1);

    int m = s1.Length;
    int n = s2.Length;

    int[] prevPrev = new int[n + 1];
    int[] prev = new int[n + 1];
    int[] curr = new int[n + 1];

    for (int j = 0; j ≤ n; j++)
        prev[j] = j;

    for (int i = 1; i ≤ m; i++)
    {
        curr[0] = i;

        for (int j = 1; j ≤ n; j++)
        {
            int cost = (s1[i - 1] == s2[j - 1]) ? 0 : 1;

            int delete = prev[j] + 1;
            int insert = curr[j - 1] + 1;
            int replace = prev[j - 1] + cost;

            curr[j] = Math.Min(Math.Min(delete, insert), replace);

            if (i > 1 && j > 1 &&
                s1[i - 1] == s2[j - 2] &&
                s1[i - 2] == s2[j - 1])
            {
                int transpose = prevPrev[j - 2] + cost;
                curr[j] = Math.Min(curr[j], transpose);
            }
        }

        int[] temp = prevPrev;
        prevPrev = prev;
        prev = curr;
        curr = temp;
    }

    return prev[n];
}
```

рис. 3 – оптимизированная реализация алгоритма Дамерау-Левенштайна

```
--------------------------------------------------------------------------------
алгоритмизация          алгоримизация          Левенштейн      1
алгоритмизация          алгоримизация          Дамерау-Лев     1
алгоритмизация          алгоримизация          Дамерау-Лев (опт) 1
--------------------------------------------------------------------------------
реализация              реолизоция             Левенштейн      2
реализация              реолизоция             Дамерау-Лев     2
реализация              реолизоция             Дамерау-Лев (опт) 2
--------------------------------------------------------------------------------
транспозиция            траспозиция            Левенштейн      1
транспозиция            траспозиция            Дамерау-Лев     1
транспозиция            траспозиция            Дамерау-Лев (опт) 1
--------------------------------------------------------------------------------
конкатенация            конкотенация           Левенштейн      1
конкатенация            конкотенация           Дамерау-Лев     1
конкатенация            конкотенация           Дамерау-Лев (опт) 1
--------------------------------------------------------------------------------
```

рис. 4 – демонстрация работы алгоритмов

Далее я провел следующий тест: я хотел посмотреть на разницу времени работы алгоритмов на одном примере. Ниже приведен результат:

Строка1	Строка2	Алгоритм	Результат	Время (мкс)
пример	1пример	Левенштейн	1	8873,000
пример	1пример	Дамерау-Лев	1	213,300
пример	1пример	Дамерау-Лев (опт)	1	198,600
пример	при1мер	Левенштейн	1	1,500
пример	при1мер	Дамерау-Лев	1	1,200
пример	при1мер	Дамерау-Лев (опт)	1	0,900
пример	пример1	Левенштейн	1	1,000
пример	пример1	Дамерау-Лев	1	1,200
пример	пример1	Дамерау-Лев (опт)	1	0,800
пример	12пример	Левенштейн	2	1,200
пример	12пример	Дамерау-Лев	2	1,200
пример	12пример	Дамерау-Лев (опт)	2	0,900
пример	при12мер	Левенштейн	2	1,000
пример	при12мер	Дамерау-Лев	2	1,100
пример	при12мер	Дамерау-Лев (опт)	2	0,800
пример	пример12	Левенштейн	2	1,100
пример	пример12	Дамерау-Лев	2	1,100
пример	пример12	Дамерау-Лев (опт)	2	0,700
пример	1при2мер3	Левенштейн	3	1,500
пример	1при2мер3	Дамерау-Лев	3	1,200
пример	1при2мер3	Дамерау-Лев (опт)	3	1,000
приМер	прМир	Левенштейн	2	0,900
приМер	прМир	Дамерау-Лев	2	0,800
приМер	прМир	Дамерау-Лев (опт)	2	0,700
ИВАНОВ	БАННВО	Левенштейн	4	0,900
ИВАНОВ	БАННВО	Дамерау-Лев	4	1,000
ИВАНОВ	БАННВО	Дамерау-Лев (опт)	4	0,800
ПРИМЕРЫ	ПРЕДМЕТ	Левенштейн	4	1,100
ПРИМЕРЫ	ПРЕДМЕТ	Дамерау-Лев	4	1,200
ПРИМЕРЫ	ПРЕДМЕТ	Дамерау-Лев (опт)	4	1,000
программирование	програмирование	Левенштейн	1	4,000
программирование	програмирование	Дамерау-Лев	1	4,500
программирование	програмирование	Дамерау-Лев (опт)	1	3,700
алгоритмизация	алгоримизация	Левенштейн	1	2,900
алгоритмизация	алгоримизация	Дамерау-Лев	1	3,500
алгоритмизация	алгоримизация	Дамерау-Лев (опт)	1	2,700

рис. 5 – сравнение времени работы алгоритмов

Вывод: оптимизированная реализация алгоритма поиска расстояния Дамерау-Левенштайна работает в любом случае быстрее.